Review Techniques

Slides copyright © 1996, 2001, 2005, 2009 by Roger S. Pressman

Also Copyright P. Johnson and IEEE.

Outline

- What are and why do reviews
- What are the benefits of reviews
- A selection of review types
- Properties of a generic review
- The IEEE gets involved
- Metrics and miscellaneous

What Are Reviews?

- a meeting conducted by technical people for technical people
- a technical assessment of a work product created during the software engineering process
- a software quality assurance mechanism
- a training ground

What Reviews Are Not

- A project summary or progress assessment
- A meeting intended solely to impart information
- A mechanism for political or personal reprisal!

Formal Technical Review

What is Formal Technical Review (FTR)?



Definition (Philip Johnson)

A method involving a structured encounter in which a group of technical personnel analyzes or improves the quality of the original work product as well as the quality of the method.

Software Quality Improvement

- Improve the quality of the original work
  - Find defects early (less costly)
  - Reduce defects
- Leads to improved productivity
  - Benefits by reducing rework build throughout the project requirements→ design→ coding → testing

Software Quality Improvement (2/4)

- Survey regarding when reviews are conducted
  - Design or Requirements: 40%
  - Code review: 30%
- Code reviews pay off even if the code is being tested later (Fagan)

Software Quality Improvement (3/4)

Improve the quality of the method

- Improve team communication
- Enhance team learning

Software Quality Improvement (4/4)

- Which impacts overall quality the most?
  - To raise the quality of the finished product
  - To improve developer skills





Key Process Areas of CMMI

Maturity Level	Key Process Area
1: Initial	None (random code bots)
2: Repeatable	Requirements Management, Software Project Planning, Software Project Tracking and Oversight, Software Subcontract Management, Software Quality Assurance, Software Configuration Management
3: Defined	Organization Process Focus, Organization Process Definition, Training Program, Integrated Software Management, Software Product Engineering, Intergroup Coordination, **Peer Reviews**
4: Managed	Quantitative Process Management, Software Quality Management
5: Optimizing	Defect Prevention, Technology Change Management, Process Change Management

Peer Reviews and CMMI

- Does not dictate specific techniques, but instead requires that:
  - A written policy about peer reviews is required
  - Resources, funding, and training must be provided
  - Peer reviews must be planned
  - The peer review procedures to be used must be documented

Inspection, Walkthrough or Review?

An **inspection** is '*a visual examination of a software product to detect and identify software anomalies, including errors and deviations from standards and specifications*'

A **review** is '*a process or meeting during which a software product is presented to project personnel, managers, users, customers, user representatives, or other interested parties for comment or approval*'

Inspection, Walkthrough or Review? (2/2)

A **walkthrough** is '*a static analysis technique in which a designer or programmer leads members of the development team and other interested parties through a software product, and the participants ask questions and make comments about possible errors, violation of development standards, and other problems*'

Source: IEEE Std. 1028-1997

Families of Review Methods

Method Family	Typical Goals	Typical Attributes
Walkthroughs	Minimal overhead Developer training Quick turnaround	Little/no preparation Informal process No measurement Not FTR!
Technical Reviews	Requirements elicitation Ambiguity resolution Training	Formal process Author presentation Wide range of discussion
Inspections	Detect and remove all defects efficiently and effectively	Formal process Checklists Measurements
		Verify phase

Source: Johnson, P. M. (1996). Introduction to formal technical reviews.

Informal vs. Formal

- Informal
  - Spontaneous
  - Ad-hoc
  - No artifacts produced
- Formal
  - Carefully planned and executed
  - Reports are produced

In reality, there is also a middle ground between informal and formal techniques

Outline

- What are and why do reviews
- What are the benefits of reviews
- A selection of review types
- Properties of a generic review
- The IEEE gets involved
- Metrics and miscellaneous

Cost-Benefit Analysis

- Fagan reported that IBM inspections found 90% of all defects for a 9% reduction in average project cost
- Johnson estimates that rework accounts for 44% of development cost
- Finding defects, finding defects early and reducing rework can impact the overall cost of a project

Cost of Defects



What is the impact of the annual cost of software defects in the US?

$59 billion

- Estimated that $22 billion could be avoided by introducing a best-practice defect detection infrastructure

Source: NIST, The Economic Impact of Inadequate Infrastructure for Software Testing, May 2002

Industry Experience with FTR

- Aetna Insurance Company:
  - FTR found 82% of errors, 25% cost reduction.
- Bell-Northern Research:
  - Inspection cost: 1 hour per defect.
  - Testing cost: 2-4 hours per defect.
  - Post-release cost: 33 hours per defect.
- Hewlett-Packard
  - Est. inspection savings (1993): $21,454,000
- IBM (using Cleanroom)
  - C system software
  - No errors from time of first compile.

Cost of Defects

- Gilb project with jet manufacturer
- Initial analysis estimated that 41,000 hours of effort would be lost through faulty requirements
- Manufacturer concurred because:
  - 10 people on the project using 2,000 hours/year
  - Project is already one year late (20,000 hours)
  - Project is estimated to take one more year (another 20,000 hours)

Jet Propulsion Laboratory Study

- Average two hour inspection exposed four major and fourteen minor faults
- Savings estimated at $25,000 per inspection
- Additional studies showed the number of faults detected decreases exponentially by phase
  - Detecting early saves time and money

Cost Benefit of Reviews

Reviews reduce rework.

- **Rework accounts for 44% of development cost!**
  - **Requirements (1%)**
  - **Design (12%)**
  - **Coding (12%)**
  - **Testing (19%)**

Reviews are *pro-active* tests.

- **Find errors not possible through testing.**

Reviews are training.

- **Domain, corporate standards, group**

Quality Improvement

- Reviews can find 60-100% of all defects.
- Reviews are technical, not management.
- Review data can assess/improve quality of:
  - Work product.
  - Software development process.
  - Review process itself.

Quality Improvement

- Reviews reduce total project cost, but have non-trivial cost (~15%).
- Early defect removal is 10-100 times cheaper.
- Reviews distribute domain knowledge, development skills, and corporate culture.

Software Inspections



Why are software inspections not widely used?

- Lack of time
- Not seen as a priority
- Not seen as value added (measured by loc)
- Lack of understanding of formalized techniques
- Improper tools used to collect data
- Lack of training of participants
- Pits programmer against reviewers

Twelve Reasons Conventional Reviews are Ineffective

1. The reviewers are swamped with information.
2. Most reviewers are not familiar with the product design goals.
3. There are no clear individual responsibilities.
4. Reviewers can avoid potential embarrassment by saying nothing.
5. The review is a large meeting; detailed discussions are difficult.
6. Presence of managers silences criticism.

Twelve Reasons Conventional Reviews are Ineffective

7. Presence of uninformed reviewers may turn the review into a tutorial.
8. Specialists are asked general questions.
9. Generalists are expected to know specifics.
10. The review procedure reviews code without respect to structure.
11. Unstated assumptions are not questioned.
12. Inadequate time is allowed.

From class website: sw-inspections.pdf (Parnas)

Outline

- What are and why do reviews
- What are the benefits of reviews
- A selection of review types
- Properties of a generic review
- The IEEE gets involved
- Metrics and miscellaneous

Michael Fagan's Contributions

- Design and code inspections to reduce errors in program development (1976)
- A systematic and efficient approach to improving programming quality
- Continuous improvement: reduce initial errors and follow-up with additional improvements
- Beginnings of formalized software inspections

Fagan's Six Major Steps

1. Planning
2. Overview
3. Preparation
4. Examination
5. Rework
6. Follow-up



Can steps be skipped or combined?

How many people hours are typically involved?

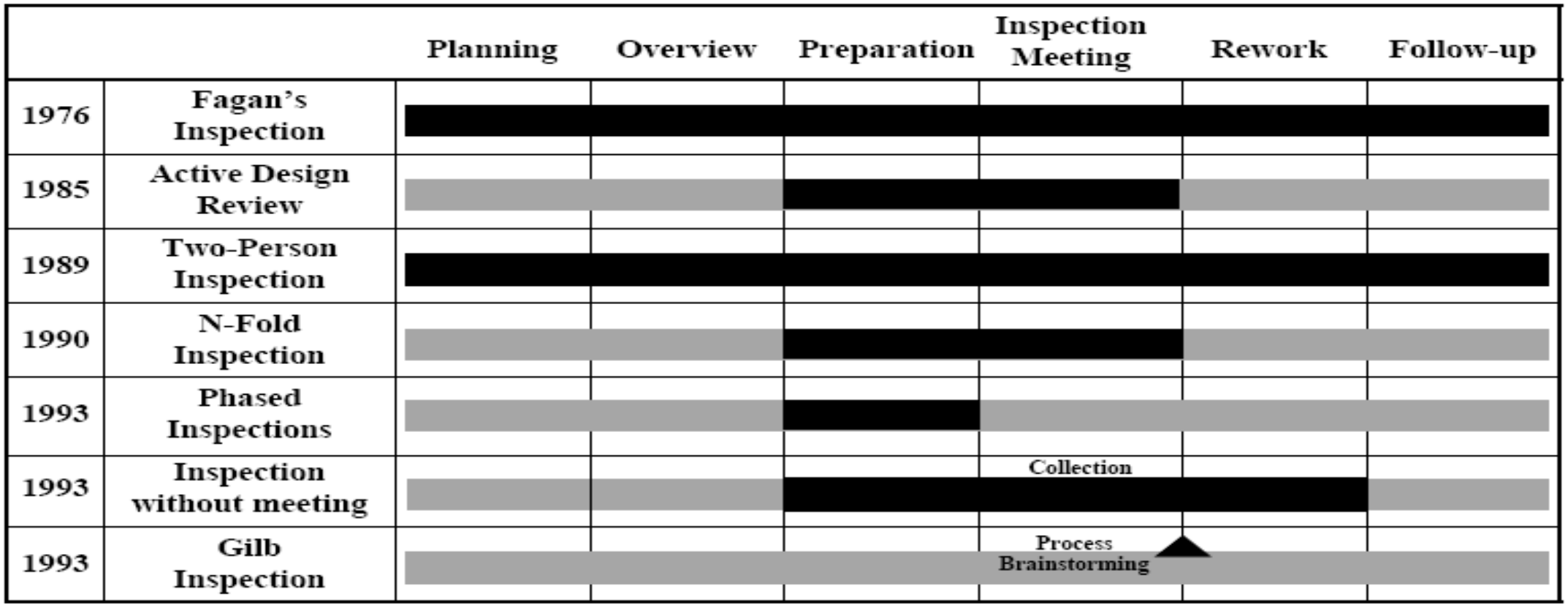
Fagan's Six Major Steps (2/2)

1. Planning: Form team, assign roles
2. Overview: Inform team about product (optional)
3. Preparation: Independent review of materials
4. Examination: Inspection meeting
5. Rework: Author verify defects and correct
6. Follow-up: Moderator checks and verifies corrections

Fagan's Team Roles

- Fagan recommends that a good size team consists of four people
- Moderator: the key person, manages team and offers leadership
- Readers, reviewers and authors
  - Designer: programmer responsible for producing the program design
  - Coder/ Implementer: translates the design to code
  - Tester: write, execute test cases

Common Inspection Processes



Active Design

- Parnas and Weiss (1985)
- Rationale
  - Reviewers may be overloaded during preparation phase
  - Reviewers lack of familiarity with goals
  - Large team meetings can have drawbacks
- Several brief reviews rather than one large review
- Focus on a certain part of the project
- Used this approach for the design of a military flight navigation system

Two Person Inspection

- Bisant and Lyle (1989)
- One author, one reviewer (eliminate moderator)
- Ad-hoc preparation
- Noted immediate benefits in program quality and productivity
- May be more useful in small organizations or small projects

N-fold Inspection

- Martin and Tsai (1990)
- Rationale
  - A single team finds only a fraction of defects
  - Different teams do not duplicate efforts
- Follows Fagan inspection steps
- N-teams inspect in parallel with results
- Results from teams are merged
- After merging results, only one team continues on
- Team size 3-4 people (author, moderator, reviewers)

Phased Inspection

- Knight and Myers (1993)
- Combines aspects of active design, Fagan, and N-fold
- Mini- inspections or “phases” with specific goals
  - Use checklists for inspection
  - Can have single-inspector or multiple-inspector phases
- Team size 1-2 people

Inspection without Meeting

- Research by Votta (1993) and Johnson (1998)
  - *Does every inspection need a meeting?*
- Builds on the fact that most defects are found in preparation for the meeting (90/10)
- Is synergy as important to finding defects as stated by others?
- Collection occurs after preparation
- Rework follows

Gilb Inspections

- Gilb and Graham (1993)
- Similar to Fagan inspections
- Process brainstorming meeting immediately following the inspection meeting

Inspection, Walkthrough or Review?

- Some researchers interpret Fagan's work as a combination of all three
- Does present many of the elements associated with FTR
- FTR may be seen as a variant of Fagan inspections (Johnson, Tjahjono 1998)

True FTR is well-defined

- Well-defined process
  - Phases (orientation, etc.)
  - Procedures (checklists, etc.)
- Well-defined roles
  - Moderator, Reviewer, Scribe, Author, etc.
- Well-defined objectives
  - Defect removal, requirements elicitation, etc.
- Well-defined measurements
  - Forms, consistent data collection, etc.